

IRE Assignment 2 Design Note

Learning from Click-Logs on EB-NeRD and MIND

Iman Kalyan Chakraborty - 2026801003

September 8, 2026

<https://github.com/astrion-coder/cs4406m26-assignment1c1/>

1 Introduction

[TODO: What A2 adds on top of A1’s retrieval pipeline: behavioural features from the click-logs, a trained two-stage re-ranker, a reproduced NRMS baseline with a paired-bootstrap ablation, and a serving/scale analysis. Both datasets, large tracks, both Codabench leaderboards.]

2 Design

2.1 Click-History and Session Features (Q1)

[TODO: The fifteen features and where each comes from; recency weighting as real elapsed time on EB-NeRD versus an ordinal proxy on MIND; the five columns MIND never had; the behaviour-window boundary and why the current impression’s own read time and scroll depth are deliberately excluded.]

2.2 The Re-Ranker (Q2)

[TODO: Option A (GBDT/LightGBM) over Option B (small neural ranker), and why: NRMS is Q3’s baseline, so choosing the neural ranker here would have collapsed the improvement and the thing it is measured against into one model. Training hosted on Kaggle, inference local. The learning curve that settled the 400k training cap.]

Candidate set: why re-ranking operates on the in-view set

So Stage 1 genuinely feeds Stage 2 — just through “how well does BM25 score this article for this user” rather than “is this article in the user’s global top-200”. That is the same `score_inview` adapter A1 built, reused unchanged, which is why the two-stage framing still holds.

[TODO: Build the argument that leads into the paragraph above: the top-200 is scored against the whole catalogue (125,541 / 104,151 articles) while re-ranking must operate on `article_ids_inview` (~12 EB-NeRD, ~37 MIND); recall@200 of 0.2–3% caps any metric the second stage could produce; both Codabench leaderboards require a permutation of the in-view set, so a ranking of our own top-200 is not submittable at all; and the two membership features that do encode top-200 overlap carry exactly zero split gain on both datasets, at 2.4–4.3× chance overlap. Note this fork was identified and documented during A1 (SPEC.md Q4 §1), not retrofitted for A2.]

2.3 Baseline Reproduced, Then Beaten (Q3)

[TODO: NRMS reproduction from the ebnerd-benchmark repo (EB-NeRD) and recommenders-team/recommenders (MIND), trained on Kaggle and exported to ONNX for local inference. The principled improvement, the ablation isolating it, and the paired bootstrap 95% CI.]

2.4 Serving and Scale Analysis (Q4)

[TODO: Index memory, p99 latency for candidate generation plus feature assembly plus re-ranker scoring, and a back-of-envelope cost per 1000 queries at a target SLA.]

2.5 Code Optimizations

[TODO: The measured memory work: the Arrow-to-Python materialization class of bug found six times over, peak stacking, the streaming join that replaced two eager collects, and BM25 determinism via `sorted(set(...))`. List format is appropriate here - this section is a changelog.]

3 Discussion

3.1 Results

[TODO: Consolidated table: AUC, MRR, nDCG@5, nDCG@10 before and after re-ranking, both datasets, val and test, with bootstrap CIs and the paired CI on each difference. Plus the sample-representativeness check against A1's full-population baselines.]

3.2 Why the Gain Differs Between the Two Datasets

[TODO: The substantive finding: EB-NeRD gains ~ 0.10 AUC against MIND's 0.006–0.016, because seven of fifteen features carry zero split gain on MIND versus two on EB-NeRD - and the single most informative EB-NeRD feature, `freshness_hours` at 48% of total gain, is one of the five MIND never had. Same model, same features, same hyperparameters; the ceiling is set by what the log records.]

3.3 Ablation and Statistical Significance

[TODO: Ablation results with the paired bootstrap CI, and the Q9 with/without serving-time-features run.]

3.4 Where This Breaks at 10x

[TODO: Feature-store joins at scale, brute-force cosine similarity needing a real ANN index, ranker-inference batching. Extend A1's genuine memory-exhaustion incident with what the re-ranking and serving stages specifically add.]

4 Leaderboard Submissions

[TODO: Screenshots from both Codabench competitions.]